When Do Dense Process Rewards Help Agentic RL? A Verifiable-Potential Criterion

Bojie Li
Pine AI

Noah Shi
University of Washington

Abstract

Agentic reinforcement learning inherited its reward from reasoning models: one bit at the trajectory's end saying whether the task was solved. It discards what the environment uniquely offers—every tool call returns a machine-checkable result—and goes blind when it matters most: on the all-fail groups that dominate early training, GRPO sees identical rewards, zero advantage, a dead update. Rewarding the process is the obvious fix but treacherous—a penalty satisfiable by doing nothing yields a degenerate *compliance* optimum, and a learned progress critic is simply hacked. We ask *when* a process reward helps and give a criterion: **it helps if and only if it is an un-gameable, verifiable potential strictly finer than the outcome whose intermediate values the policy can reach**—a property of the *domain*, checkable before training.

We validate it across five settings. Where it says *yes*: a verifiable progress signal in theorem proving eliminates dead updates (0 vs. 16 of 40 iterations, 3 seeds); an un-gameable penalty in system administration cuts destructive actions $\sim 4\times$ at *equal* task success. Where it says *no*: customer-service *intent* is the reward itself, not a rule, so process rewards neutralize harm but never beat outcome-only; in software repair the finer signal is unreachable—across 156 rollouts the policy makes zero partial progress—so it gives no gradient. A controlled aligned-to-misaligned sweep reproduces the verdict arm by arm. One rule explains every gain and failure, and tells a practitioner, before spending compute, whether a process reward is worth adding.

1 Introduction

Outcome-only reinforcement learning carried reasoning models from a base checkpoint to sophisticated behaviour with no supervised cold-start [DeepSeek-AI, 2025, Shao et al., 2024]. The recipe is simple: sample a group of trajectories, reward the ones that solve the task, and push the policy toward them with a group-relative advantage. The agentic era inherited it wholesale. A tool-using agent is trained by rewarding the trajectories that complete the task, and nothing else.

But an agentic trajectory is not a chain of thought. It is a sequence of *interactions with an environment*, and the environment is itself a *verifier*: error codes, tool results, and state transitions are machine-checkable intermediate signal that reasoning RL never had. You cannot verify a chain-of-thought step mid-stream without redoing it. Outcome-only RL discards this signal, and pays twice. The first cost is *gradient*, and it is the lens we use throughout. A group-relative advantage is, mechanically, a within-group variance: GRPO centres each reward on the group mean [Shao et al., 2024], so a group can teach the policy only insofar as its rewards *differ*. Early

in training on a hard task most groups are *all-fail*, with every rollout missing, and a binary outcome has zero variance there: identical rewards, zero advantage, a dead update, exactly when the policy most needs shaping. A dense, verifiable *process* signal restores the variance by distinguishing the failed rollouts the outcome cannot tell apart. This is the paper in one sentence: **a useful dense process reward is the within-group variance the outcome cannot supply**. Most of what follows establishes when such a signal exists, and when, used naively, it backfires. The second cost is *safety*. The outcome reward is defined on the terminal state, so it is blind to harm *on the path*: an agent that deletes a production database and then rebuilds it ends in a correct state and is rewarded, though the irreversible damage is already done.

The intermediate signal the environment provides could fill both gaps. The trouble is that using it naively backfires, in ways that have made practitioners wary of process rewards altogether. Consider a penalty an agent can satisfy by *doing nothing*, such as "never call before looking up the account." It is maximized by an idle policy, and in the all-fail regime, where the process channel is the only gradient, the optimizer climbs straight into that degenerate *compliance* optimum and task success collapses to zero. A *learned* progress critic, the natural way to get a dense signal without writing rules, simply moves the reward-hacking up one level: the policy learns to fool the critic. Process rewards sometimes give large gains and sometimes cause catastrophic collapse, and the field has lacked a way to tell the two cases apart in advance.

This paper. We give that way. We argue that whether a dense process reward helps is not a property of the algorithm but of the *domain*, and we make it precise with a single criterion (Figure 1):

> *A dense process reward improves agentic RL if and only if it is an **un-gameable, verifiable proxy for progress**: a potential strictly finer than the terminal outcome (so it varies across the failed rollouts in an all-fail group), whose cheapest maximizer must still solve the task (so the optimizer cannot game it), and whose intermediate values the policy can **actually reach** (so the variation is realized, not merely possible).*

The three conditions (finer-than-outcome, un-gameable, reachable) are each individually testable *before* training, by inspecting the domain and a handful of base-policy rollouts. The criterion is grounded in potential-based reward shaping, which guarantees that any potential leaves the optimal policy unchanged [Ng et al., 1999]. A process reward is therefore safe to add exactly when it is such a potential, and useful exactly when it additionally supplies group-relative gradient the outcome cannot.

Validation as a predictive law. We do not propose one method and show it works. Instead we treat our whole body of results, successes *and* failures, as predictions the criterion must get right. Across five settings (Table 1) its verdict matches the measured outcome in every case, and a controlled sweep that walks one signal from aligned to misaligned reproduces that verdict arm by arm within a single experiment. The contributions below give the results, and the rest of the paper is their defence.

Contributions.

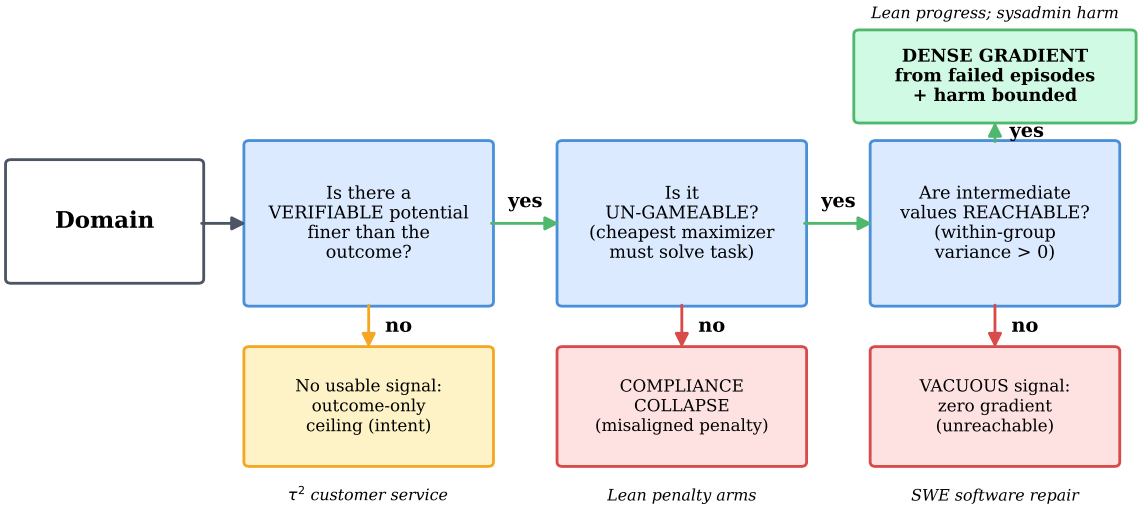

Figure 1. The verifiable-potential criterion, and the five settings it sorts. Reading left to right, a dense process reward helps only if the domain passes three gates: a verifiable potential *finer* than the terminal outcome must exist, it must be *un-gameable* (its cheapest maximizer still has to solve the task), and its intermediate values must be *reachable* by the policy. Failing the first gate leaves only the outcome ceiling (customer-service *intent*). Failing the second invites compliance collapse (a misaligned penalty). Failing the third makes the signal vacuous (software repair, where partial progress never occurs). Passing all three yields dense gradient from failed episodes and a bounded harm path (theorem proving and system administration).

- **A criterion** (§2) that predicts, before training, whether a dense process reward will help in a given domain. It unifies the all-fail-group blindness of group-relative RL, the compliance attractor of misaligned penalties, and the failure of learned critics under one condition: an un-gameable, reachable, verifiable potential finer than the outcome.
- **A controlled demonstration that the criterion is predictive** (§3). A three-seed un-gameability sweep at 30B walks a single Lean signal from aligned to misaligned, and the policy's fate tracks admissibility: penalty-free progress signals survive, a pure penalty reliably collapses, adding a discharge to the penalty makes survival possible but high-variance, and outcome-gating a gameable signal gives the most consistent survivor. A granularity/sparsity study shows the gain scales with the potential's fineness. And an SWE reachability probe in which all 156 rollouts score zero partial progress shows a structurally-finer potential that is empirically vacuous.
- **Two positive instantiations** where the criterion says yes. First, process rewards make outcome learning *sample-efficient*, eliminating dead updates (0 vs. 16 of 40 iterations over 3 seeds) and reaching target success several times faster (§4–5). Second, an un-gameable penalty achieves *harm reduction on an axis the outcome cannot see*: $\sim 4\times$ fewer destructive actions at *equal* task success (§6).
- **Three boundaries** where it says no, mapped honestly. There is an intent ceiling on a real benchmark where verifiable rules cover policy *validity* but not task *intent* (§7). There is a learned self-critic shown to relocate rather than remove the problem (§8). And there is a discovery ceiling where no on-policy reward escapes a never-sampled precondition (§9).

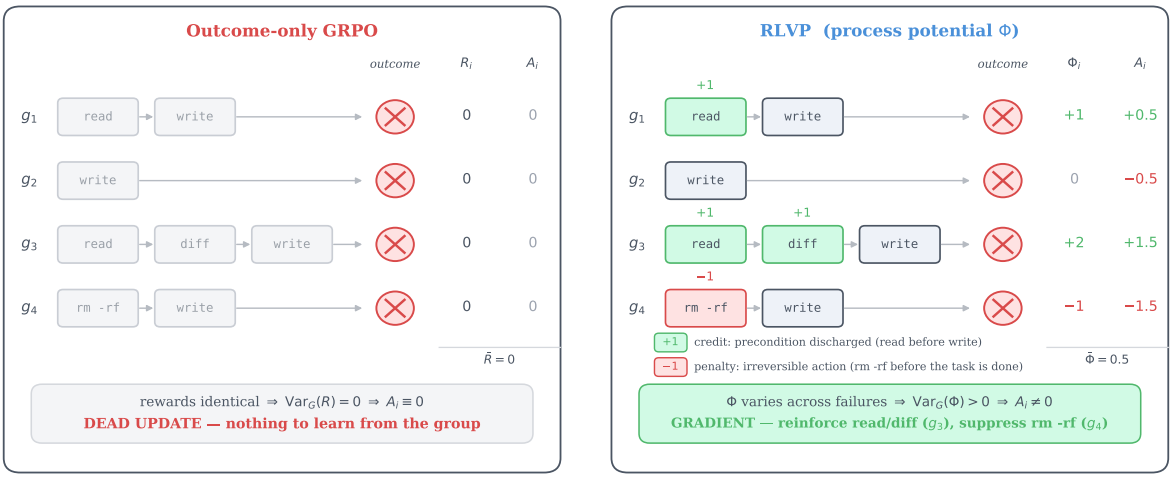

Figure 2. Why a process signal can help where the outcome cannot. A worked all-fail group: four rollouts $g_1 \dots g_4$ of an agent editing a config, every one ending in task failure (outcome 0). *Left (outcome-only GRPO):* the only score is the terminal outcome, identical ($R_i = 0$) across the group, so the group-mean-centred advantage $A_i = R_i - \bar{R}$ is zero on every rollout, a dead update with nothing to learn. *Right (process potential Φ):* a verifiable potential credits a discharged precondition (`read` before `write`, $+1$) and penalizes an irreversible action (`rm -rf` before the task is done, -1), separating the same four failures: $\Phi = (+1, 0, +2, -1)$, $\bar{\Phi} = 0.5$, so the advantage $A_i = \Phi_i - \bar{\Phi}$ is now non-zero. It reinforces the careful rollout g_3 (read, then diff) and suppresses the destructive g_4. The condition is exactly that the process signal *vary within the group*: a potential finer than the outcome, so $\mathrm{Var}_G(\Phi) > 0$ where $\mathrm{Var}_G(R) = 0$.

2 A Criterion for When Process Rewards Help

The gradient gap (why one would want a process reward). An all-fail group (every rollout scoring zero) has no reward spread, so a group-relative method yields no gradient there, a *dead update* (Figure 2). These groups dominate early training on long-horizon tasks, exactly when shaping matters most. A process reward that turns on *how* an episode behaves rather than *whether* it succeeds varies across those failed rollouts, and is gradient where the outcome has none. Two things can still go wrong.

The two failure modes (why one cannot use just any process reward). In the all-fail regime the process channel is the *only* gradient, so if the cheapest way to maximize it is anything other than solving the task, the optimizer finds it. Two cases recur. A **penalty** is almost always gameable: its cheapest maximizer is inaction (never act ⇒ never violate), so on all-fail batches it drives the policy into a degenerate compliance-only optimum, where success goes to zero while "compliance" is perfect. A **learned critic** is gameable in a subtler way: defining progress by a learned proxy relocates the credit-assignment problem into the proxy, which the policy then learns to fool. Neither failure is about tuning. Both are structural.

The criterion. The gradient gap and the two failures share a root, which is the criterion stated in §1: a process reward is admissible only as an un-gameable verifiable potential finer than the outcome, and useful only if the policy also reaches its intermediate values. Three conditions, each checkable before training:

(1) **Finer than outcome.** The signal Φ takes more values than the binary terminal reward, and those values *vary across the rollouts in an all-fail group*. (A uniform Φ is centered out by GRPO and supplies no gradient.)

(2) **Un-gameable.** The cheapest policy that maximizes Φ must still solve the task. Operationally, name the cheapest policy that maximizes Φ *without* solving the task. If it is degenerate (idle, padding, error-avoidance by inaction), Φ is inadmissible.

(3) **Reachable.** The policy's rollouts actually attain intermediate Φ values: there is realized within-group variance, not merely a potential that exists on paper.

These three conditions are not independent. They are the three terms of one quantity. Writing each rollout's reward as its outcome plus a weighted potential, $R_i = O_i + \beta\,\Phi_i$, the only thing a group-relative method extracts from a group is the spread of its rewards,

$$\mathrm{Var}_G(R) \;=\; \mathrm{Var}_G(O) \;+\; \beta^2\,\mathrm{Var}_G(\Phi) \;+\; 2\beta\;\mathrm{Cov}_G(O,\Phi),$$

and the gates are its terms. The outcome term vanishes on all-fail groups. The potential term is non-zero only when Φ is both finer than the outcome *and* reached, which is conditions (1) and (3) together, $\mathrm{Var}_G(\Phi) > 0$. And the cross term, whether that variation points toward success, is what condition (2) protects. (The method's per-channel normalisation and per-token placement, §5, rescale and route this signal but leave which terms vanish, and the sign of the alignment, unchanged.) Potential-based shaping [Ng et al., 1999] certifies that any such Φ is *safe*: it leaves the optimal policy unchanged. The variance reading says which is *useful*. A verifiable progress credit (Lean's goal count strictly decreasing, a failing test turning green) can satisfy all three conditions. A bare penalty fails (2). A learned critic fails verifiability and usually (2). A domain whose only finer signal is unreachable fails (3). Table 1 previews how the five settings we study fall out.

3 The Criterion as a Predictive Law

If the criterion is more than a restatement of intuition, it should make *advance predictions* that a controlled experiment confirms. We give three such tests here. The per-domain training results that follow (§4 onward) then show the same verdicts in the wild.

An un-gameability sweep: admissibility predicts survival. We construct five process signals for Lean theorem proving on miniF2F [Zheng et al., 2022], spanning aligned to misaligned, and pre-register each one's cheapest gaming policy and the criterion's verdict. We then train Qwen3-30B-A3B [Team, 2025] with each, three seeds, holding everything else fixed (Figure 3). The result tracks admissibility, and three seeds sharpen exactly where the criterion is sharp and where it is not. The *penalty-free* signals reliably **survive** on every seed: a gameable "any-valid-tactic" credit (the most stable arm), an aligned goal-progress discharge, and that gameable credit *gated on the eventual outcome*, which is un-gameable by construction and gives the most consistent survivor. The *pure penalty* (an errored-tactic penalty with no discharge) reliably **collapses** on every seed to essentially zero: its cheapest maximizer, avoid errors by not attempting, is the compliance attractor, and with no positive signal to escape it the policy dies every time. The revealing case is the *penalty-plus-discharge* arm: single-seed it looked dead, but across three seeds it is *bimodal*, collapsing on two seeds and rescued all the way to

Table 1. The criterion sorts five settings, and its verdict matches the measured outcome in every one. Each row is a domain. The three middle columns are the criterion's gates: does a verifiable potential finer than the outcome exist, is it un-gameable, and are its intermediate values reachable. "✓/✗" marks the gate that decides the row. The criterion predicts *help* only when all three pass.

Setting	Domain / model	Finer Φ?	Un-game?	Reach?	Verdict	Observed
Theorem proving (progress)	Lean miniF2F 30B, synth. 4B	✓	✓	✓	**help**	dead updates $\to$ 0, dense gradient
System admin (harm)	TerminalBench 4B	✓	✓	✓	**help** (harm axis)	$\sim$4$\times$ fewer violations, equal success
Lean signal sweep (controlled)	miniF2F 30B	varies	**varies**	✓	per arm	pure penalty dies, penalty-free survive
Customer service (intent)	τ-bench airline 4B	✗	—	—	**no help**	rules match, never beat outcome
Software repair	SWE-bench 30B	1/3 only	—	✗	**no help**	0% solve, all rollouts Φ=0

perfect success on the third, the largest variance of any arm. So adding a verifiable discharge to a misaligned penalty does not *reliably* save it. It makes survival possible but unstable. The robust law is therefore narrower and cleaner than a single seed suggested: **penalty-free progress signals survive, a pure penalty reliably collapses, and outcome-gating yields the most consistent survivor**, each called by the criterion in advance.

Granularity and sparsity: the gain scales with the potential's fineness. A complementary, lower-variance test isolates condition (1). On a synthetic N-stage chained task with a verifiable potential equal to the number of completed stages, we vary the potential's *granularity* (coarse = outcome, versus one milestone, versus every stage) and the outcome's *sparsity* (N). A potential strictly finer than the outcome drives success where the outcome alone is essentially zero, and the advantage *grows* as the outcome becomes sparser. That is exactly the regime where all-fail groups dominate and the finer potential has the most uniquely-available gradient to offer. This is the positive, controlled face of the criterion: fineness is not incidental, it is the mechanism.

A reachability probe: a finer potential that is vacuous. Conditions (1) and (2) are about the signal. Condition (3) is about the *policy*, and it is the one that quietly fails in hard domains. We test it directly on SWE-bench software repair [Jimenez et al., 2024], where the natural potential is the fraction of the hidden `FAIL_TO_PASS` tests an episode makes pass. Two facts emerge (Figure 4). Structurally, the finer potential barely exists: two-thirds of instances have a single failing test, so Φ collapses to the binary outcome, and only a third are "multi-test" with any room for partial progress. Empirically, even that third is *unreachable*: across 156 rollouts of a 30B policy, *every* episode scores $\Phi = 0$. The model never makes a single hidden test pass, let

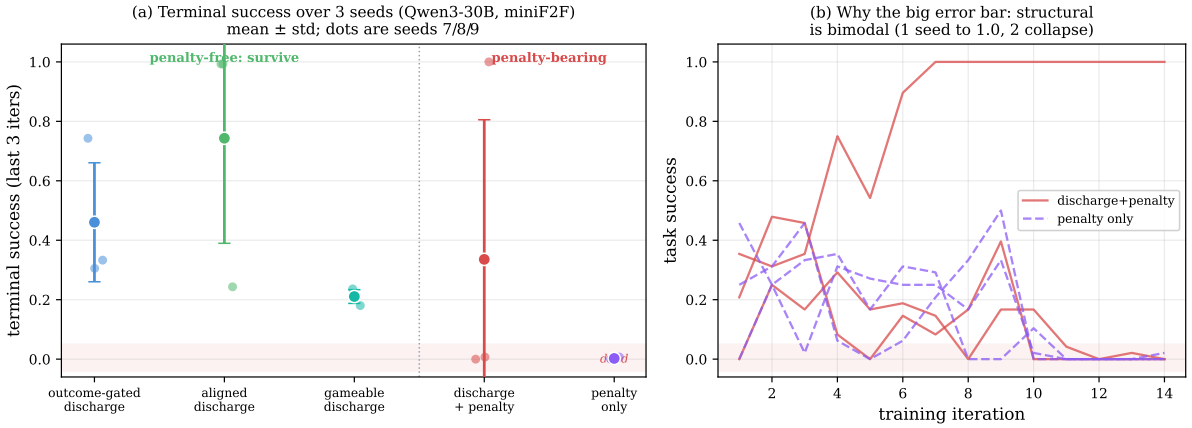

Figure 3. The criterion predicts which signals survive (Qwen3-30B, miniF2F, 3 seeds). *(a)* Terminal success (last-3-iteration mean) for five process signals, mean $\pm$ std with the three seed values as dots. The three *penalty-free* signals (left of the divider) sit well above the dead zone on every seed, while the *pure penalty* (right) is reliably dead. The *discharge + penalty* arm has by far the largest error bar. *(b)* Why: that arm is *bimodal*, collapsing on two seeds but rescued by the goal-progress discharge to 1.0 on one, whereas the pure penalty collapses on all three. The robust reading is *penalty-free survives, pure penalty reliably collapses, gating gives the most consistent survivor*. The precise magnitudes remain noisy at 30B (§10).

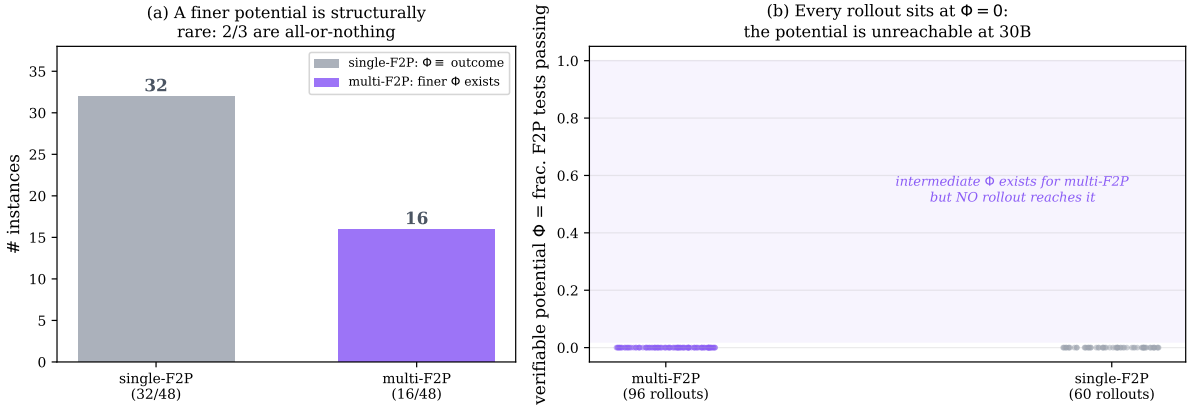

Figure 4. In software repair the finer potential is structurally rare and empirically unreachable. *(a)* Of 48 single-file-fix SWE-bench instances, two-thirds have a single failing test, so the test-fraction potential equals the binary outcome, and only one-third admit a finer Φ. *(b)* For those, a 30B policy's rollouts never realize it: all 156 rollouts (multi- and single-test) score $\Phi = 0$. The shaded band of intermediate values exists but is empty: zero within-group variance, hence zero gradient. The criterion's third gate, reachability, is what fails here.

alone a proper subset. The within-group variance is exactly zero, so a Φ-based reward would be centered out to no gradient. This is why the dense SWE signal fails, and the criterion locates the cause precisely: not gameability, but *unreachability*. There is no realized intermediate state for any reward, gameable or not, to act on.

4 Where the Criterion Says Yes, I: Sample-Efficiency in Theorem Proving

When the verifiable signal *is* the bottleneck to success (read before write, decrease the goal count, pass the gate), the process channel and the outcome point the same way, and the criterion predicts a large efficiency gain. We confirm it on a controlled suite of long-horizon agentic tasks, where every claim about gradient is exact rather than estimated. The testbed is a family of *N-stage chained tasks*: an agent completes N independent file-operations subtasks in one episode using tools `list_dir`, `read_file`, `write_file`, `delete`, `run_tests`, `submit`. Success requires all N stages, so N tunes base difficulty smoothly (N=4 sits near 8% base success). We instantiate the process signal as **Reinforcement Learning from Verifiable Penalties** (RLVP): a penalty when a call violates a verifiable rule (overwrite a file never read, or submit without testing a change) and a credit when a call *discharges* a pending obligation (reads before writing, tests after mutating). These are *specifications*, written once, requiring no solved trajectories. They are the analog of R1-Zero's verifier, not of a supervised cold-start. The credit is carried on a separate channel with its own normalizer and attached to the *responsible tokens*. All experiments train Qwen3-4B [Team, 2025] with our own GRPO, and efficiency is always counted in *episodes generated* so that resampling costs are visible.

Dead updates are pervasive, and only a reachable, admissible signal removes them. We count the fraction of training iterations whose update is *dead*, an all-fail group with no reward spread (Figure 5). On the chained task, outcome-only GRPO is dead on 65% of iterations. DAPO [Yu et al., 2025], built precisely to dodge all-fail groups by resampling, only reaches 54% and pays a 5.6× generation tax to get there. An admissible process reward cuts dead updates to 8% at no extra sampling, by producing gradient from the very failures the outcome channel reduces to zero. The cure is the credit scheme, not the budget, but it is contingent on *reachability*. On the silent-precondition gate (§9), where the pivotal action is never sampled, *every* reward-only method is dead almost always: outcome and DAPO on 100% of iterations, and even RLVP on 76%, because a discharge can only fire on behaviour the policy actually attempts. There the dead updates are broken not by a denser reward but by a single demonstration. Dead updates are not a rounding error, and not just any process reward removes them.

The gain is large and consistent, while DAPO's is neither. Figure 6 plots held-out success against episodes generated. Outcome-only GRPO crawls, spending most of its budget on dead updates, then converges only at the end. RLVP rises immediately by learning from the early all-fail groups. Reimplemented dense-process baselines (GiGPO-style step advantages [Feng et al., 2025], StepTool-style per-call rewards [Yu et al., 2024]) also far outpace outcome-only: the broad lesson is that *any* admissible dense signal helps. Aggregated over three seeds (Figure 7), RLVP reaches the success threshold several times faster than GRPO with *near-zero* seed variance, while outcome-only GRPO is a lottery governed by when it first stumbles into a non-all-fail group. DAPO [Yu et al., 2025], designed for the same all-fail problem, *discards* such groups and resamples. We measure a 5.6× sampling tax that leaves it, on the fair axis of episodes generated, slower than vanilla GRPO. Confronted with an all-fail group, DAPO says "no information, discard," while RLVP says "no *outcome* information, but plenty of *process* information." Breaking the binary-reward degeneracy is exactly what lets RLVP keep the failures DAPO must throw away.

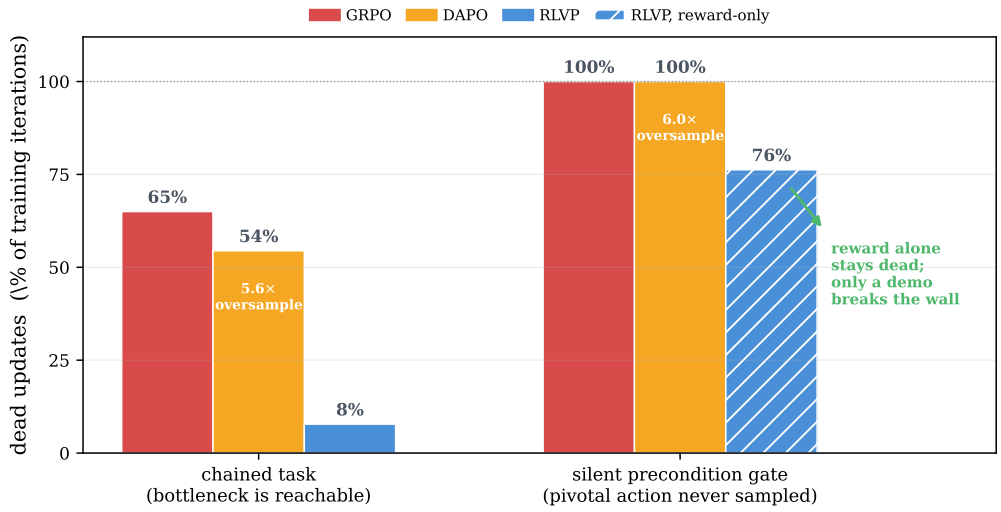

Figure 5. Dead updates across two regimes and three credit schemes. Fraction of training iterations whose update is dead, an all-fail group with zero reward spread (Qwen3-4B, 3-seed mean on the chained task). *Chained task*, where the bottleneck behaviour is reachable: outcome-only GRPO is dead 65% of the time, DAPO resamples away all-fail groups but still sits at 54% and pays a 5.6× generation tax, and the admissible process reward (RLVP) cuts dead updates to 8%. *Silent precondition gate* (§9), where the pivotal action is never sampled: outcome and DAPO are dead on *every* iteration, and even reward-only RLVP (hatched) is dead 76% of the time, because no reward can credit a behaviour the policy never attempts, and only a demonstration seed breaks the wall. The cure for dead updates is not density but a *reachable, admissible* signal.

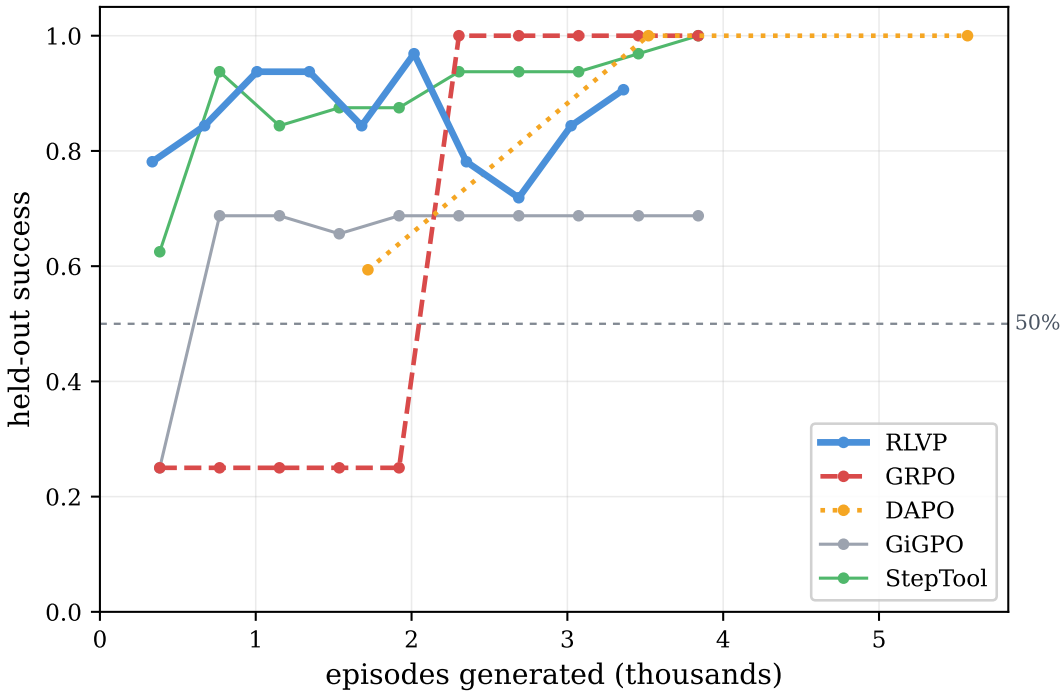

Figure 6. Sample efficiency on the calibrated hard task (N=4): held-out success vs. episodes generated. Outcome-only GRPO sits near the base rate for most of training, then converges only at the end. RLVP climbs immediately by learning from the all-fail groups. Dense-process baselines also far outpace outcome-only. The x-axis counts episodes *generated*, the fair currency that includes DAPO's resampling.

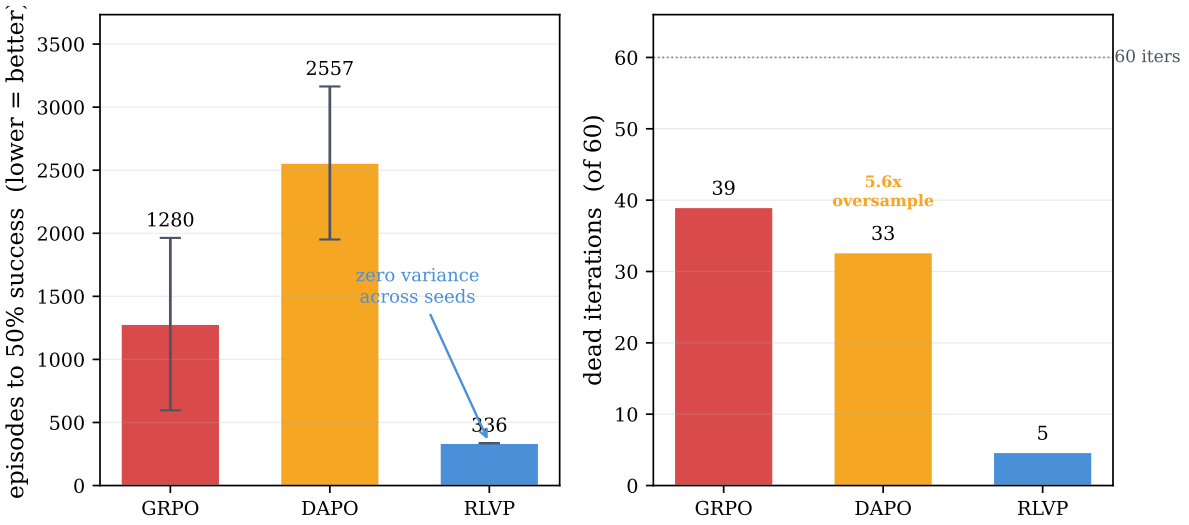

Figure 7. Headline efficiency comparison over three seeds. *Left:* episodes to the success threshold (lower is better). RLVP is several times faster than GRPO and DAPO and, unlike both, has near-zero seed variance. GRPO's large error bar is the all-fail lottery. *Right:* the mechanism. GRPO and DAPO burn most iterations on dead or stalled updates, and DAPO additionally generates 5.6× the episodes it uses.

The gain holds at 30B on a real benchmark. The chained tasks are synthetic and the policy is 4B. Does the efficiency survive a capable model on a real task? We repeat the comparison on hard miniF2F theorems [Zheng et al., 2022] with Qwen3-30B, using the aligned goal-progress potential against outcome-only with everything else fixed (Figure 8). The verifiable potential reaches full success in five iterations. Outcome-only takes twelve, after stalling near zero through its first iterations, the all-fail dead-zone the potential skips. Both eventually saturate (the set is learnable given budget), so the claim is not that outcome-only fails but that the potential is more than twice as fast and avoids the early dead phase, exactly where the gradient gap bites. This is the criterion's *yes* verdict at scale: a task hard enough that all-fail groups dominate, yet whose progress signal is *reachable*. This is the regime where a dense reward has the most uniquely-available gradient to offer.

5 An Anatomy of the Gain

Which component does the work? Ablating on $N{=}4$ (Figure 9) revises some intuitions. **The token-attached channel is the efficiency lever**: folding the same rule rewards into the scalar return doubles the episodes to threshold. Placing credit on the responsible tokens, not merely having a dense reward, is what buys the speed, and is the operational reason per-channel (not global) normalization is non-negotiable. **Annealing protects the ceiling**: removing it does not slow learning but lowers final success, because unrelaxed process pressure drifts the policy toward over-compliant, bloated episodes, a mild version of the compliance attractor that returns with force in §7. **The discharge credit is the positive signal that escapes all-fail groups**: penalties suppress wrong moves but cannot induce a never-tried right one, and removing the discharge leaves more dead iterations and a lower ceiling. Finally, **the signal can be free**: replacing every hand-written rule with three *universal* meta-rules derived only from per-tool category tags (observe/mutate/verify/terminal) and the environment's own error

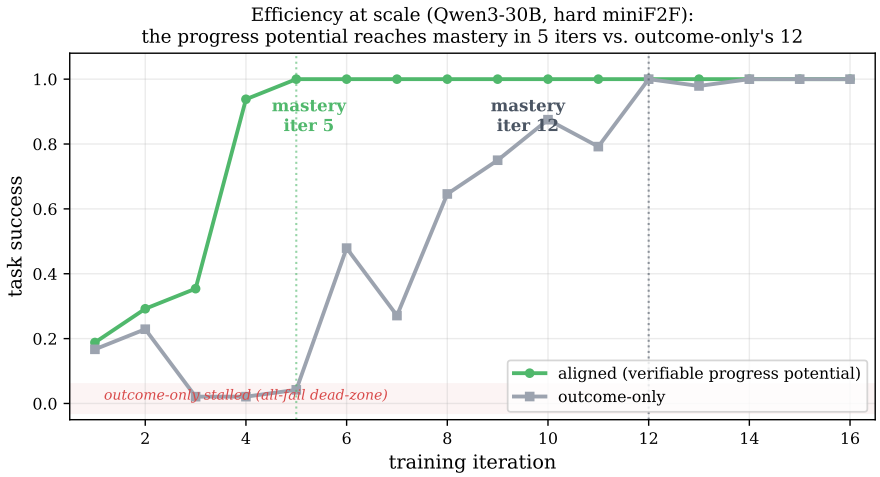

Figure 8. The efficiency gain holds at 30B on real theorems. On hard miniF2F with Qwen3-30B, the aligned verifiable-progress potential reaches full success in five iterations, while outcome-only takes twelve and stalls near zero for its first iterations (the all-fail dead-zone). Both saturate, so the gain is *speed* (more than twice as few iterations to mastery, and avoidance of the early dead phase), not a difference in final success.

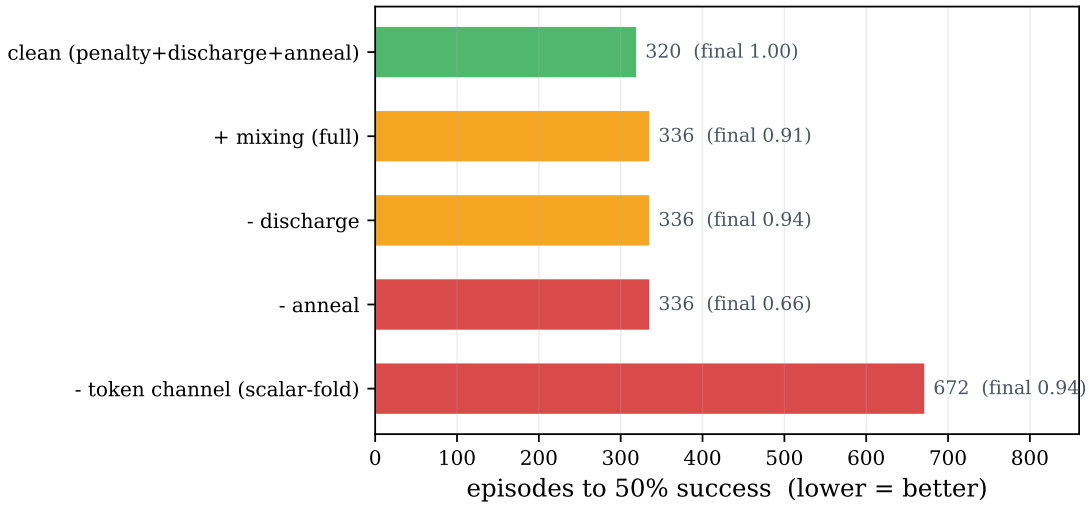

Figure 9. Component attribution on $N{=}4$ (episodes to threshold, final converged success annotated). The token-attached credit channel is the efficiency lever (folding into a scalar return doubles the cost). Annealing protects the ceiling, and the clean recipe is the best variant. A per-step length cost, helpful only on very long horizons, hurts here.

codes recovers the hand-engineered efficiency almost exactly, with no task-specific human input. RLVP's specification cost can, in well-structured environments, be paid by metadata that tool schemas already expose.

6 Where the Criterion Says Yes, II: Harm Reduction Independent of Outcome

The second place the criterion says yes is the safety use the outcome reward structurally cannot serve. Because the terminal reward is defined on the final state, it is blind to *irreversible harm*

Table 2. Harm reduction at equal outcome (TerminalBench, Qwen3-4B, 3 seeds, mean $\pm$ std). Task success is identical and at the floor (4B rarely solves the benchmark), yet the un-gameable harm penalty cuts harmful actions roughly fourfold. The policy is not going passive: it takes about three times *more* productive actions while violating less. Harm lives on an axis the terminal outcome cannot see.

	RLVP (harm penalty)	**Outcome-only**
Task success	0.097 ± 0.069	0.097 ± 0.063 (equal, at floor)
Violations / episode	$\mathbf{0.95 \pm 0.71}$	3.99 ± 0.57
Productive actions / episode	$\sim$**14**	$\sim$4

on the path. A verifiable penalty on harmful actions is exactly an un-gameable bound on that path. Its cheapest maximizer, take the harmful action zero times, *is* the desired behavior, so it does not invite the compliance attractor the way an orthogonal procedural penalty does. We test this on real TerminalBench Docker tasks with Qwen3-4B, with penalties on repeat-error and blind-destructive actions (Table 2). At this scale the model rarely solves the benchmark, so *task success is identical and at the floor*. And yet RLVP commits roughly *four times fewer* harmful actions. Crucially it does not achieve this by going passive: it takes about three times *more* productive actions while violating less. This is the clean separation the criterion predicts: harm lives on an axis independent of the outcome, and an un-gameable penalty reduces it without changing whether the task is solved.

7 Where the Criterion Says No, I: Task Intent Is Not a Verifiable Rule

The hardest case for any process-reward method is a task whose difficulty is not procedural at all. We move onto τ-bench-style airline customer service [Yao et al., 2024], whose reward turns on *semantic* policy adherence (authorization, refund eligibility, confirmation) rather than the structural ordering generic rules encode. The criterion predicts the first gate will fail: there is no verifiable potential finer than the outcome, because what is missing is *intent*, which is the reward itself. The experiment (Figure 10) confirms it in three informative tiers. *Generic* structural rules, the read-before-write hygiene that works on chained tasks, are now orthogonal to the reward and actively *harm*: the policy discovers that the cheapest way never to violate "confirm before calling" is to not place the risky calls at all, and collapses into the compliance attractor within a handful of iterations. *Policy-derived procedural* rules (obtain the user id and look up the reservation before modifying it), outcome-instrumental by construction and paired with early annealing, remove the harm. A correct modification *requires* the looked-up state, so the discharge pulls toward the productive workflow rather than idleness. *Verifiable semantic* rules (do not modify a basic-economy reservation, and do not use a payment method absent from the profile) prune failure-bound actions and lift the best iterations nearly to outcome-only's peak. But neither aligned tier *beats* outcome-only on average. The coverage gradient saturates. Verifiable rules express policy *validity*, whether a modification is permissible, while the residual difficulty is *which* permissible change *this* user wants, and a rule encoding that would simply be the task specification. RLVP accelerates the policy-verifiable component of success and neutralizes harm, but cannot substitute for outcome learning of intent (Figure 11).

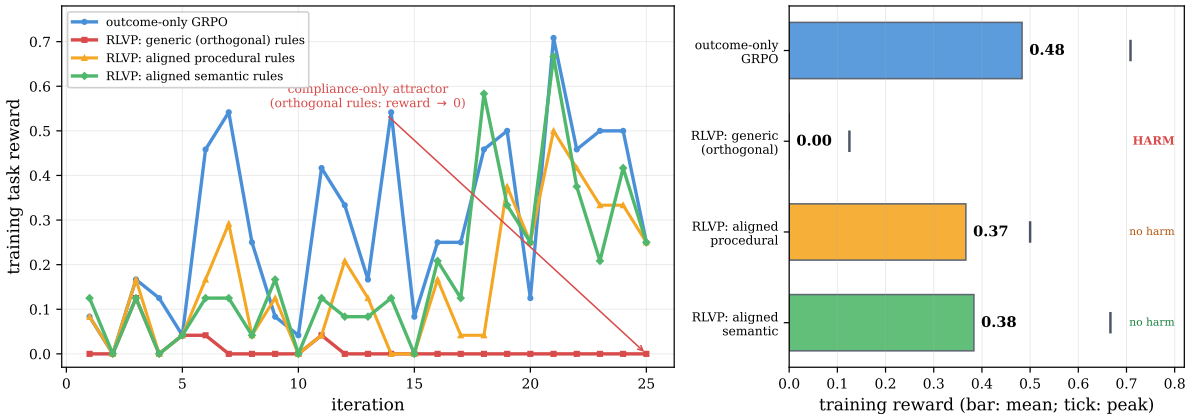

Figure 10. The coverage gradient, and where it saturates, on τ-bench airline. *Left:* training reward for four signals. Outcome-only learns the benchmark. Generic rules orthogonal to the reward collapse into the compliance attractor. Policy-derived procedural and verifiable-semantic rules (with early annealing) do not. *Right:* mean reward per tier (bars) with peak (ticks). Misaligned rules *harm*. Aligned rules *remove* the harm and the semantic peak nearly reaches outcome-only's, but neither *beats* it. The residual gap is task intent, which is the reward itself.

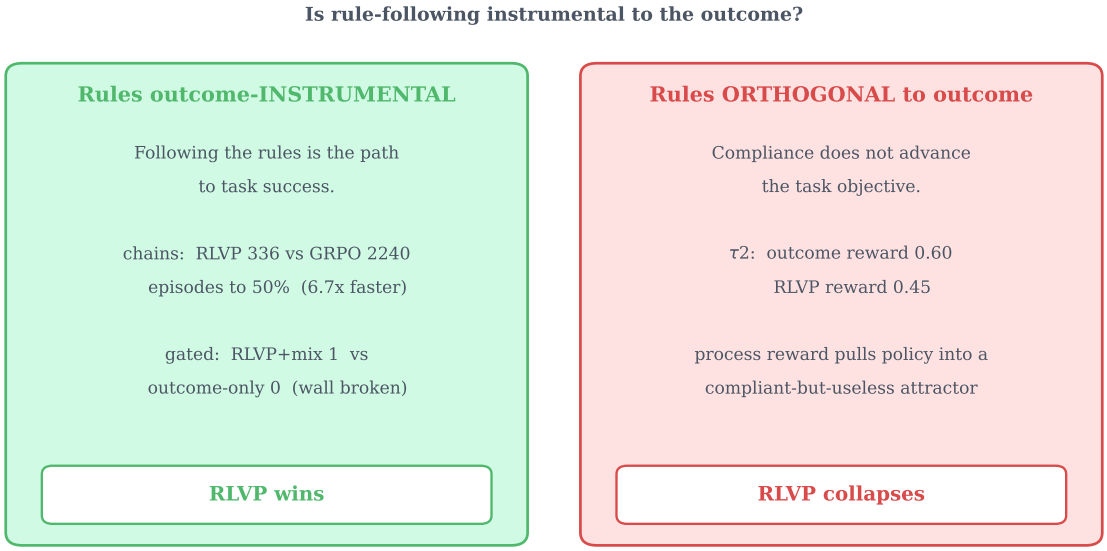

Figure 11. The boundary on a single axis: are the rules outcome-instrumental? When the verifiable rules are aligned with what success requires (chained and gated tasks: the workflow *is* the task), process rewards convert compliance into large outcome gains. When orthogonal to the reward (τ-bench with generic rules), the same machinery optimizes compliance into a degenerate policy.

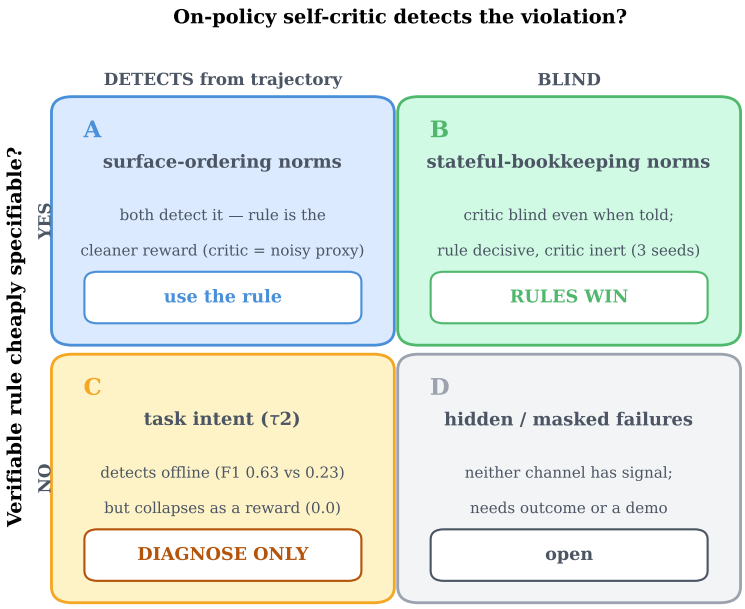

Figure 12. When a verifiable rule beats a same-model self-critic, and when neither trains. Axes: whether a verifiable rule is cheaply specifiable, and whether the on-policy critic can detect the violation. The learned critic is a poor training reward in every quadrant. Its only robust use is offline diagnosis at the intent frontier.

8 Where the Criterion Says No, II: A Learned Critic Relocates the Hack

If verifiable rules cannot reach intent, why not let the policy *judge itself*, in the lineage of self-rewarding LMs and RLAIF [Yuan et al., 2024, Lee et al., 2023, Bai et al., 2022, Zheng et al., 2023]? We test this with the critic as the *same model* as the policy (no larger judge), so any signal is genuine on-policy self-reflection rather than distillation, mapping outcomes across two axes: whether a verifiable rule is cheaply specifiable, and whether the on-policy critic can detect the violation (Figure 12). As a *detector*, blind self-critique flags violations evident from the trajectory surface but is essentially blind to *stateful-bookkeeping* norms (did it read *this* file before overwriting it) even when handed the rules, and the blind spot is structural: per-rule recall stays near zero as the critic is scaled up. As a *reward* in the all-fail regime (Table 3), a deterministic rule with identical credit structure is decisive on every seed, while the self-critic, *with perfect recall* against the rule oracle, is inert. A *frozen* critic fails identically, isolating the cause as the critic's small but nonzero false-positive rate, not its non-stationarity. At the intent frontier (Table 4), the self-critic is a useful *offline* detector, far outscoring the verifiable rules as a failure predictor, yet *collapses* when used as the training reward. This is the criterion applied to a *learned* signal: defining progress by a learned proxy relocates the credit-assignment problem into the proxy, which is then either too imprecise to optimize against or, at the intent frontier, reduces to approximating the value function itself. Verifiability is not a convenience. It is what makes the dense channel safe to optimize.

Table 3. A deterministic rule beats a same-model self-critic as a dense reward, in the all-fail regime (consumer-service domain, 1.7B, three seeds, late-training mean ± std). Rule and critics share the identical penalty-only credit structure, and the critic additionally has *perfect recall* against the rule oracle. The rule is decisive every seed, while the critics are inert. The *frozen* (stationary) critic fails like the live one, isolating the cause as the critic's ∼6% false-positive imprecision rather than non-stationarity.

process reward	detection (P/R)	violations/episode	task success
rule penalty (deterministic)	1.00/1.00	$\mathbf{0.38 \pm 0.44}$	$\mathbf{0.33 \pm 0.47}$
self-critic, live (co-evolving)	0.94/1.00	0.94 ± 0.05	0.00
self-critic, frozen (stationary)	0.94/1.00	0.93 ± 0.02	0.00

Table 4. Self-critique is a usable intent *detector* but a failed intent *reward* (τ-bench airline, Qwen3-4B). *Left:* as a failure predictor on rule-clean (intent) failures the self-critic far exceeds the verifiable semantic rules (3 rollout seeds). *Right:* as the dense training reward the same self-critique collapses, while outcome-only and the rules both learn (training reward, early→late, 20 iterations).

offline: failure prediction		*trained as reward*	
channel	F1	channel	reward (early→late)
semantic rules	0.23 ± 0.06	outcome-only	$0.09 \rightarrow \mathbf{0.50}$
blind self-critique	$\mathbf{0.63 \pm 0.02}$	semantic rules	$0.10 \rightarrow 0.35$
		blind self-critique	$0.01 \rightarrow \mathbf{0.00}$

9 Where the Criterion Says No, III: The Discovery Ceiling of Self-Evolution

A third boundary is about *reachability* (the criterion's condition (3)), and it is the one that quietly governs hard domains. Sample efficiency is a statement about *speed* to a ceiling the chained tasks eventually reach. To ask whether a process reward can raise the *ceiling*, we need a task the model cannot saturate, and one that is hard to *discover* rather than hard to *compute* (process rewards guide procedure, not answers). We build a *silent precondition gate*: to write a protected file the agent must first read an access-control file and request access. Skip either and the write silently no-ops: it reports success but does nothing, and the task fails with no hint why. Calibration confirms the trap is total: the base model never reads the access file, even when the rule is in its prompt. The result (Figure 13) is sharper than "RLVP wins." *Every* reward-only method (outcome, DAPO, clean RLVP) fails to zero, because the discharge credit for reading the access file can only reward that behavior if the policy *samples* it, and it never does. A single rule-synthesized demonstration, injected through the process channel, breaks the wall and drives the policy to perfect, generalizing success where prompting the same behavior had failed. When the required behavior is never sampled, no on-policy reward can induce it, and imitation must seed what reward then grows. This is the same reachability wall the software-repair probe (§3) hit from the other side. It maps cleanly onto R1-Zero (discoverable workflow: reward-only suffices) versus R1 (un-discoverable precondition: a demonstration cold-start is necessary).

10 Discussion

The criterion as a design tool. The contribution is less a method than a decision procedure. Before adding a dense process reward, a practitioner can ask three questions in turn and

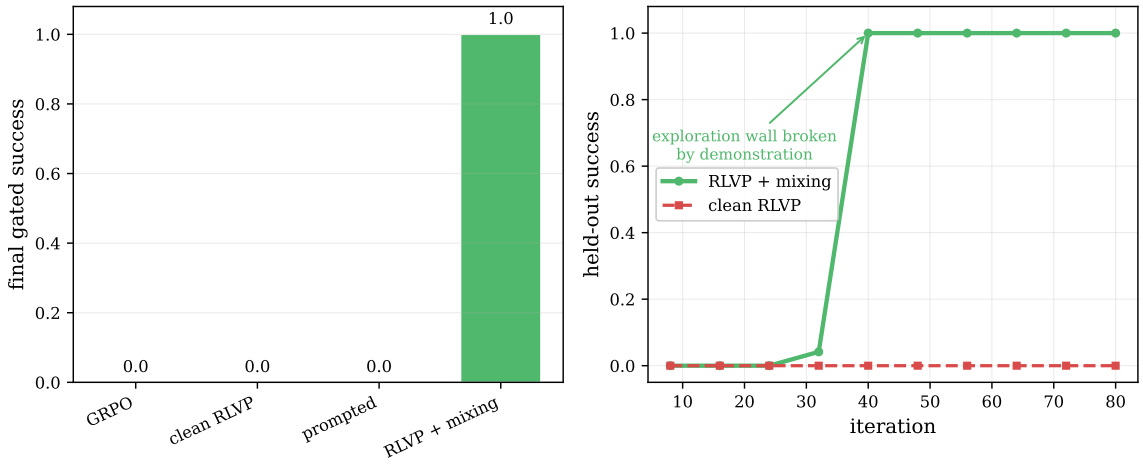

Figure 13. A non-saturating ceiling test, and the discovery wall. *Left:* on the silent-gate task every reward-only method (outcome, DAPO, clean RLVP, even rules-in-prompt) converges to zero, because the required precondition is never sampled and the discharge credit can never fire. *Right:* a single synthesized demonstration injected through the process channel breaks the wall to perfect held-out success, a sharp phase transition, while clean RLVP stays at the floor.

predict the result. Does a verifiable potential finer than the outcome exist? Is its cheapest maximizer forced to solve the task? Do base-policy rollouts reach its intermediate values? The five settings here were not cherry-picked successes. They were chosen to span the gates, and the criterion's verdict matched in every case, including the failures. When the answer to all three is yes, a verifiable progress credit is the right tool and the gains are large. When a penalty is the only available signal, it belongs on the *harm* axis (bounding the path), never as the learning gradient in an all-fail regime. When the residual difficulty is intent, only the outcome can teach it.

A systems note: verifiable agentic RL is often verifier-bound. The property that makes a process reward cheap, a machine-checkable environment, tends to make the verifier a *CPU* process (a proof kernel, a test runner, a container). In our 30B theorem-proving runs the GPU generated a tactic in milliseconds and then waited seconds on the Lean kernel, leaving GPU compute utilization in the single digits at full memory (Figure 14). This is workload-dependent, not a law. Generation-heavy or learned-verifier settings are GPU-bound. But for the verifiable domains where our criterion most often says yes, throughput is gated by environment parallelism, and the right systems design co-locates many verifier workers per accelerator rather than scaling accelerators.

Limitations. The 30B un-gameability sweep is high-variance: even over three seeds the per-arm magnitudes carry large error bars (Figure 3), so we report the *survival pattern* (penalty-free survives, pure penalty reliably collapses, penalty-plus-discharge is bimodal) rather than precise values, corroborated by the lower-variance 4B granularity/sparsity study and the τ-bench collapse. The variance itself carries a lesson we did not anticipate. A single seed made the penalty-plus-discharge arm look reliably dead, when three seeds reveal it is occasionally rescued, a caution against reading a single agentic-RL run as a verdict. The reachability result is measured at 30B on a benchmark that is 0%-solve for this model. A stronger model that

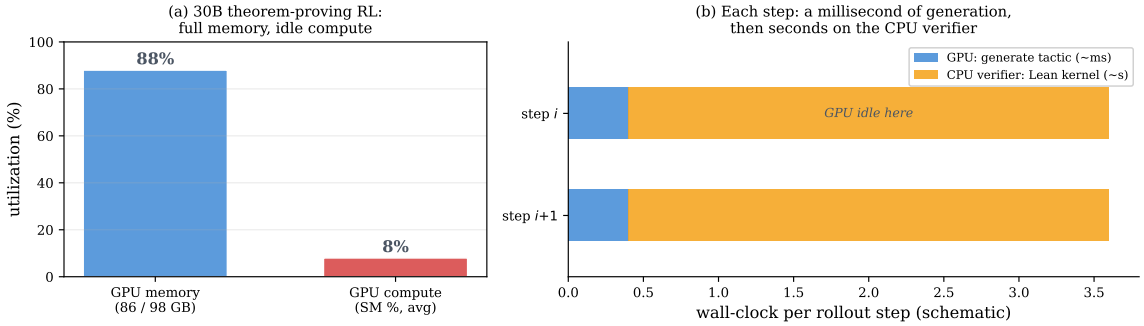

Figure 14. Verifiable agentic RL is often verifier-bound, not compute-bound. *(a)* During 30B theorem-proving RL the GPU runs at full memory but single-digit compute utilization. *(b)* Why: each rollout step spends a millisecond generating a tactic on the GPU, then seconds in the CPU-side Lean kernel verifying it, during which the GPU idles. Throughput then scales with verifier (environment) parallelism rather than accelerators. This holds for cheap CPU verifiers, though generation-heavy or learned-verifier workloads remain GPU-bound.

achieved partial fixes could make the SWE potential non-vacuous, so the claim is explicitly conditional on reachability. And our verifiable potentials are hand-identified per domain. Automating their discovery is the natural next step.

11 Related Work

Outcome RL and credit assignment. GRPO [Shao et al., 2024] and the R1 line [DeepSeek-AI, 2025] established outcome-only verifiable-reward RL as the dominant recipe. The long-horizon credit-assignment difficulty has prompted step-level value estimation and turn-level shaping [Feng et al., 2025, Yu et al., 2024]. We frame the problem as the *all-fail-group blindness* of group-relative methods and characterize when a *verifiable, non-learned* process signal fills the gap. DAPO [Yu et al., 2025] is the closest direct comparison. We show its dynamic sampling relocates the all-fail cost rather than recovering the discarded signal.

Process rewards, rubrics, and reward shaping. Process reward models [Lightman et al., 2024] and step-grained tool-use rewards [Yu et al., 2024, Qian et al., 2025] attach credit before the outcome, and rubric- and verifier-based rewards [Gunjal et al., 2025] extend verifiable RL beyond auto-checkable domains. Our criterion is orthogonal: it says *when* such a dense signal is admissible. The connection to classical reward shaping is precise but the question is different. Potential-based shaping [Ng et al., 1999] certifies that any potential is *safe* (it leaves the optimal policy unchanged), whereas we ask which potential is *useful*, and answer with a property of the group rather than of the optimum: the within-group variance the outcome cannot supply, non-zero (finer, reachable) and aligned (un-gameable).

Learned rewards and self-critique. Self-rewarding LMs [Yuan et al., 2024], RLAIF [Lee et al., 2023], Constitutional AI [Bai et al., 2022], and LLM-as-judge [Zheng et al., 2023] replace rule-based reward with model judgments. Our controlled same-model study finds a learned critic dominated by a deterministic verifiable rule even at perfect recall, and collapsing where no rule applies. This is evidence that a learned proxy relocates rather than removes reward-hacking.

Demonstrations and the discovery wall. R1-Zero [DeepSeek-AI, 2025] self-evolves with no cold-start, and off-policy guidance mixes expert trajectories into on-policy RL [Yan et al., 2025]. We locate the boundary empirically: reward-only suffices on discoverable workflows, and a demonstration becomes necessary precisely when a required behavior is never sampled. Customer-service benchmarks with explicit policy documents [Yao et al., 2024] motivate our intent-ceiling analysis.

12 Conclusion

Agentic RL's environment is a verifier, offering dense intermediate signal that reasoning RL never had. But using it naively either wastes it (a uniform signal centered out by group-relative RL) or is catastrophic (a gameable penalty's compliance collapse, a hacked learned critic). We gave a single criterion that resolves when the signal helps: a dense process reward improves agentic RL if and only if it is an un-gameable, verifiable potential finer than the outcome whose intermediate values the policy can reach. The criterion is checkable before training, grounded in potential-based shaping, and predictive: across five settings it called every success and every failure, including a controlled sweep in which penalty kills and outcome-gating rescues exactly as predicted. Where it says yes, the payoff is concrete: dead updates eliminated and harm cut fourfold at equal success. Where it says no, it says so for a reason: intent is not a verifiable rule, and a structurally-finer potential is worthless if unreachable. The durable contribution is not another process-reward method. It is the line the criterion draws between the part of agentic success that a verifiable signal can densify and accelerate, and the part that only the outcome can teach, together with the one quantity behind it: a useful dense process reward is the within-group variance the outcome cannot supply.

Acknowledgements

This paper was produced using a voice-directed *whisper coding* workflow [Pine AI, 2025], in which the author specifies and reviews the work by voice while AI agents carry out the coding and experiments. AI tools including Pine Copilot and Claude Code with Claude Opus 4.8 were used during this research. We thank BSQL Networking for hosting the NVIDIA RTX PRO 6000 Blackwell Workstation Edition GPU.

References

Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022.

DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

Lang Feng et al. Group-in-group policy optimization for llm agent training. *arXiv preprint arXiv:2505.10978*, 2025.

Anisha Gunjal et al. Rubrics as rewards: Reinforcement learning beyond verifiable domains. *arXiv preprint arXiv:2507.17746*, 2025.

Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

Harrison Lee, Samrat Phatale, Hassan Mansoor, Kellie Lu, Thomas Mesnard, Colton Bishop, Victor Carbune, and Abhinav Rastogi. RLAIF: Scaling reinforcement learning from human feedback with ai feedback. *arXiv preprint arXiv:2309.00267*, 2023.

Hunter Lightman, Vineet Kosaraju, Yura Burda, et al. Let's verify step by step. *International Conference on Learning Representations (ICLR)*, 2024.

Andrew Y Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *International Conference on Machine Learning (ICML)*, pages 278–287, 1999.

Pine AI. Pine AI: The most natural human-computer interface is your voice. Blog post, 2025. URL https://www.19pine.ai/blog/pine-ai-the-most-natural-human-computer-interface-is-your-voice. Accessed 2026-06-28.

Cheng Qian et al. Toolrl: Reward is all tool learning needs. *arXiv preprint arXiv:2504.13958*, 2025.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Jianhao Yan et al. Learning to reason under off-policy guidance. *arXiv preprint arXiv:2504.14945*, 2025.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.

Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.

Yuanqing Yu et al. Steptool: Enhancing multi-step tool usage in llms through step-grained reinforcement learning. *arXiv preprint arXiv:2410.07745*, 2024.

Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Sainbayar Sukhbaatar, Jing Xu, and Jason Weston. Self-rewarding language models. *arXiv preprint arXiv:2401.10020*, 2024.

Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. minif2f: a cross-system benchmark for formal olympiad-level mathematics. *International Conference on Learning Representations (ICLR)*, 2022.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *arXiv preprint arXiv:2306.05685*, 2023.